

# Tecnología Digital VI

## Interpretación y Generación

Licenciatura en Tecnología Digital - UTDT

# Interpretabilidad

Hasta este punto vimos redes con una gran capacidad para resolver problemas reales sobre imágenes. Al complejizarse las arquitecturas y loss functions, el entendimiento fino de qué está haciendo completamente el modelo en cada punto puede que haya quedado un poco relegado.

Si predicen bien, ¿Para qué necesitamos interpretar nuestros modelos?

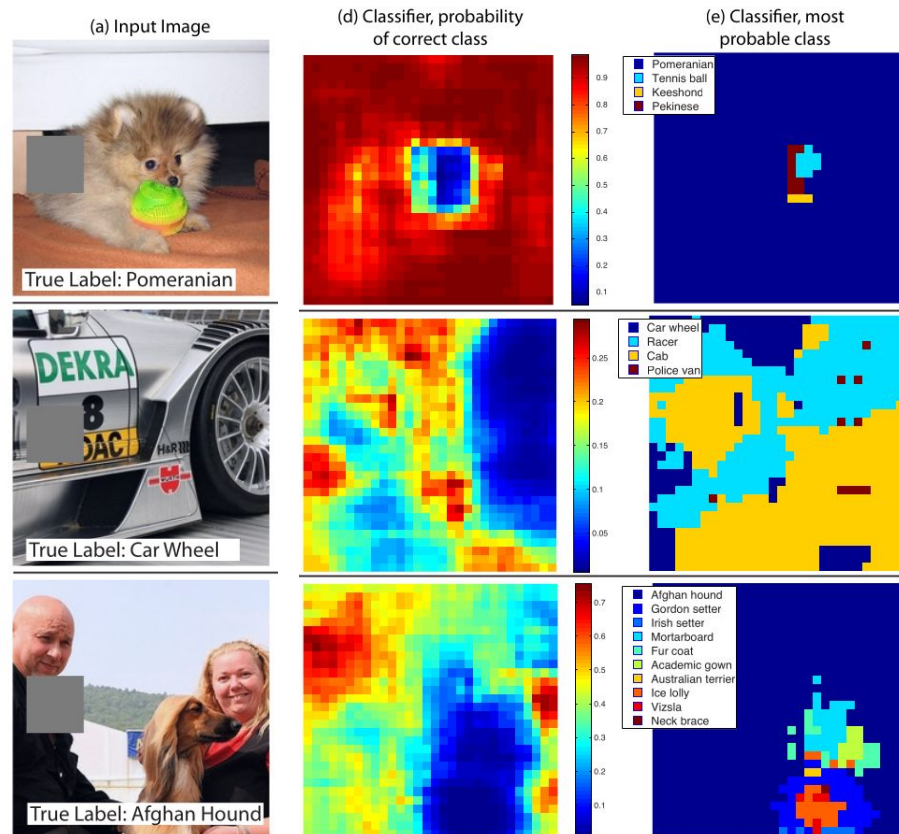
- Para explicar por qué toman ciertas decisiones sobre muestras puntuales.
- Para explicar cuáles son las características del dataset que más influye en la predicción.
- Para entender los sesgos de nuestros modelos.
- Para entender las limitaciones actuales y poder construir mejores modelos.
- Para poder manipular componentes internas del modelo en busca de diferentes objetivos.

# Interpretabilidad

Dado que es complejo diseccionar todas las composiciones de funciones que están sucediendo en una red para entender completamente la transformación final, podemos comenzar con técnicas que no miren las componentes internas sino que observen el comportamiento total de la red.

# Interpretabilidad: Sensibilidad a la oclusión

¿Cómo afecta la predicción si tapamos una cierta parte de la imagen?



# Interpretabilidad

Las CNNs son una composición compleja de funciones. Las capas exteriores (la de entrada y la de salida) están más cerca de *datos conocidos*. Podemos comenzar por ellas para tener algunas visualizaciones de lo que está pasando en la red.

La primera capa es la única que trabaja directamente sobre los píxeles de entrada y eso nos puede ayudar a entender cómo los está transformando.

La última capa es la única que trabaja directamente con la salida y eso puede ayudar a entender cómo es la transformación total de la red.

# Interpretabilidad: primera capa

Ya desde los primeros trabajos de CNNs se hizo el ejercicio de entender los filtros de la primera capa.

Los filtros son fáciles de visualizar porque el producto interno con un vector se maximiza cuando se hace la operación contra el mismo vector.

En criollo, el filtro maximiza lo que se ve exactamente igual al filtro.



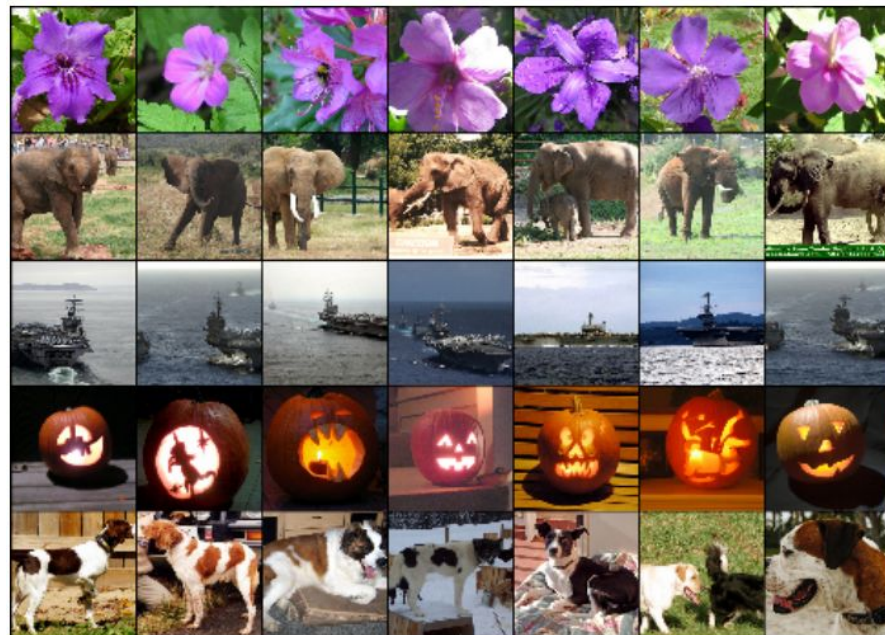
Figure 3: 96 convolutional kernels of size  $11 \times 11 \times 3$  learned by the first convolutional layer on the  $224 \times 224 \times 3$  input images. The top 48 kernels were learned on GPU 1 while the bottom 48 kernels were learned on GPU 2. See Section 6.1 for details.

Figure 3 shows the convolutional kernels learned by the network's two data-connected layers. The network has learned a variety of frequency- and orientation-selective kernels, as well as various colored blobs. Notice the specialization exhibited by the two GPUs, a result of the restricted connectivity described in Section 3.5. The kernels on GPU 1 are largely color-agnostic, while the kernels on GPU 2 are largely color-specific. This kind of specialization occurs during every run and is independent of any particular random weight initialization (modulo a renumbering of the GPUs).

# Interpretabilidad: última capa

La última capa oculta es que la muestra la transformación casi completa. En el caso de las CNNs para ImageNet, lo único que falta es la parte FC que devuelve la clasificación.

Una manera de ver qué está haciendo la red es visualizar cómo quedan las muestras en este espacio latente de features.





# Image Similarity

A veces no necesitamos clasificar nuestras muestras, sino que queremos buscar imágenes parecidas al input.





# Interpretabilidad: última capa

En ese espacio latente de features estamos en una alta dimensión y no podemos visualizar más que *proxies* como los vecinos más cercanos.

Otra opción es aplicar alguna técnica de reducción de la dimensionalidad para visualizarlo.

[https://cs.stanford.edu/people/karpathy/cnnembed/cnn\\_embed\\_full\\_4k\\_seams.jpg](https://cs.stanford.edu/people/karpathy/cnnembed/cnn_embed_full_4k_seams.jpg)



# Interpretabilidad: última capa

Algo a notar interesante de las visualizaciones de última capa en CNNs para ImageNet es que guardan una cohesión espacial de similaridad de imágenes en el espacio latente que no fue explícitamente forzada en la loss function (otras redes sí lo hacen). Es razonable que esa organización sea *útil* pero igualmente es interesante el comportamiento emergente.

En este punto la interpretabilidad sirve para modelos futuros (“ah, yo puedo inducir a la red a que tenga este comportamiento, ¿cómo?”).

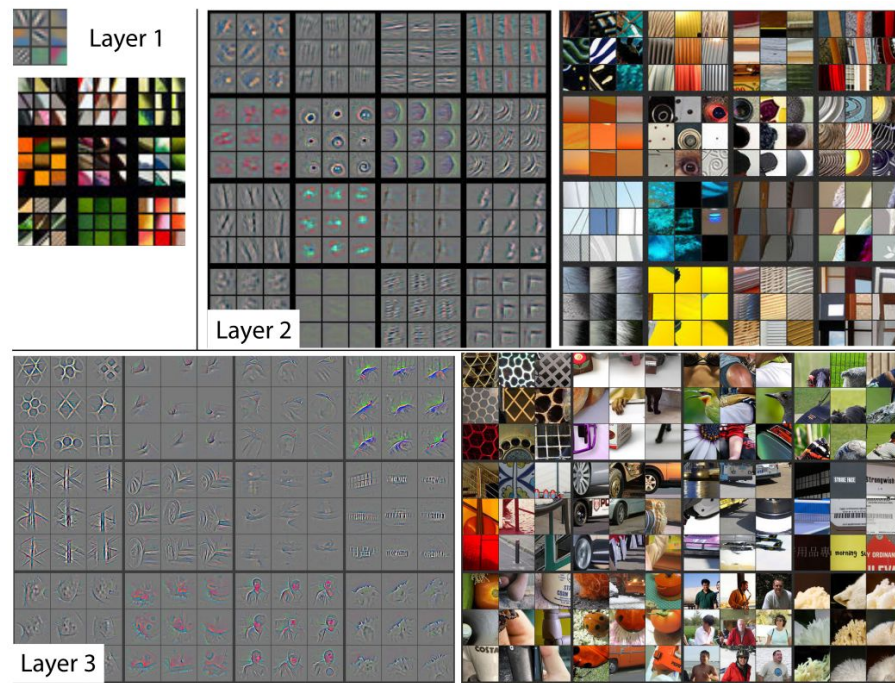
# Interpretabilidad: capas intermedias

Sobre las capas exteriores podemos hacer visualizaciones particulares por su semántica. ¿Qué se puede hacer para entender lo que está sucediendo en el medio de la red?

La visualización de filtros es posible pero más compleja y con menos semántica clara porque no trabajan directamente sobre el input. Una primera idea podría ser entonces al menos ver qué tipo de imágenes activan cada uno de los filtros en las redes entrenadas.

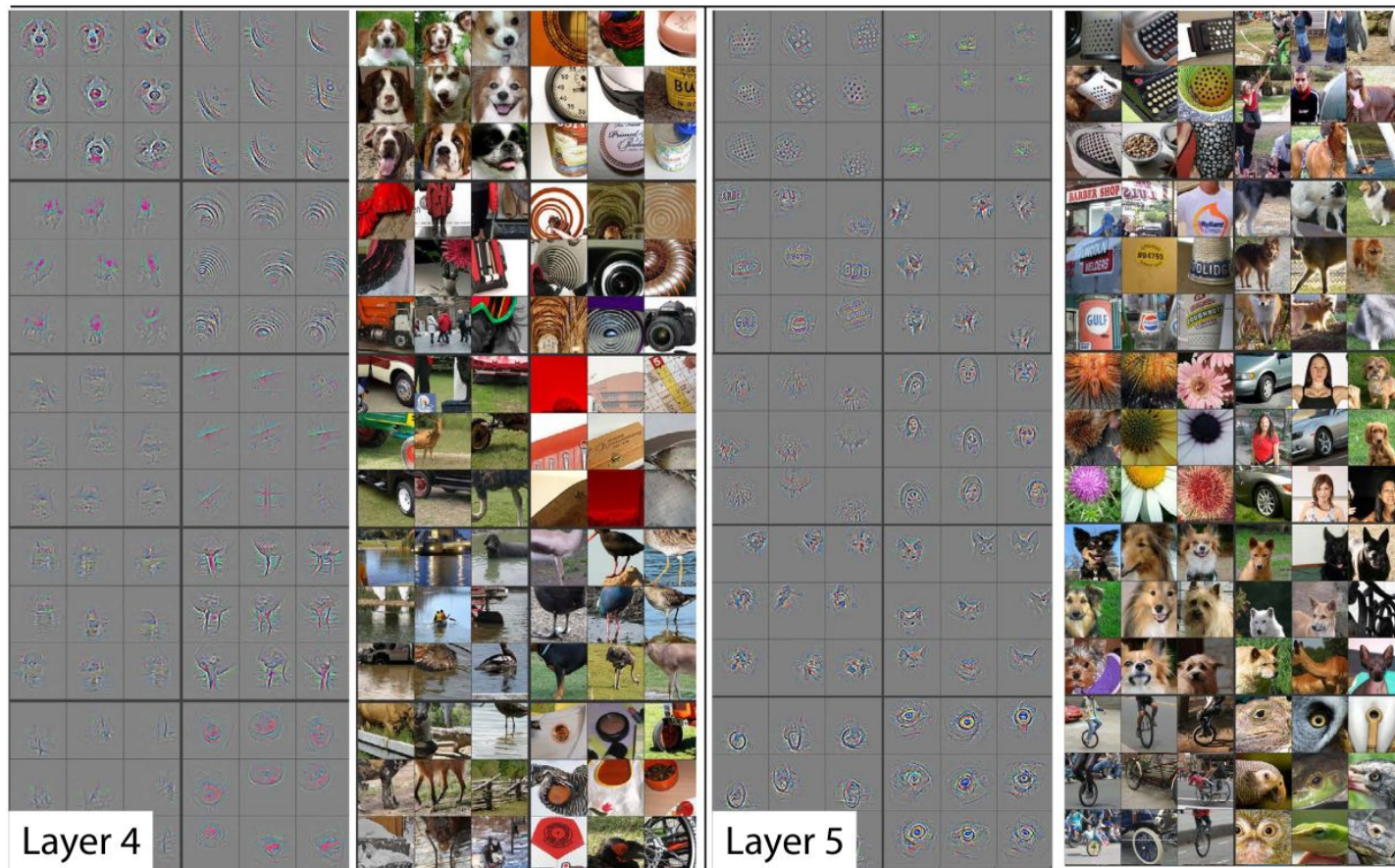
# Interpretabilidad: capas intermedias

Si no podemos visualizar un filtro, al menos podemos ver qué imágenes son las que más lo activan. Adicionalmente, es simple hacer la cuenta del tamaño y ubicación del campo receptivo del filtro elegido, así que también podemos ver exactamente qué *pedacito* de la imagen fue la que activó al filtro.





# Interpretabilidad: capas intermedias



# Saliency maps

La idea original de la sensibilidad a la oclusión era interesante y simple pero también bastante poco granular. Pasábamos de la imagen original a una que tenía un gran sector completamente tapado.

¿Qué pasa si en vez de tapar todo un sector simplemente tapamos un solo píxel? ¿Qué pasa si en vez de tapar un píxel (hacerlo 0) lo movemos un poco de su valor original?

Tengo una imagen que es de una pelota de tenis. Al pasarla por la red me dice que es *tennis ball* con 96% de probabilidad. ¿Cuánto afecta a ese número una variación puntual pequeña en cada píxel?

# Saliency maps

Presentado como un análisis de sensibilidad sobre los píxeles, puede parecer que la idea para esta visualización es hacer muchos *forward pass* con modificaciones de la imagen.

En realidad, lo que se hace es una única pasada hacia atrás.

Estamos acostumbrados a pensar las redes como una función con respecto a los pesos (y así es como tomamos gradientes y optimizamos), pero nada impide pensarla como una función con respecto a los píxeles de la imagen de entrada.

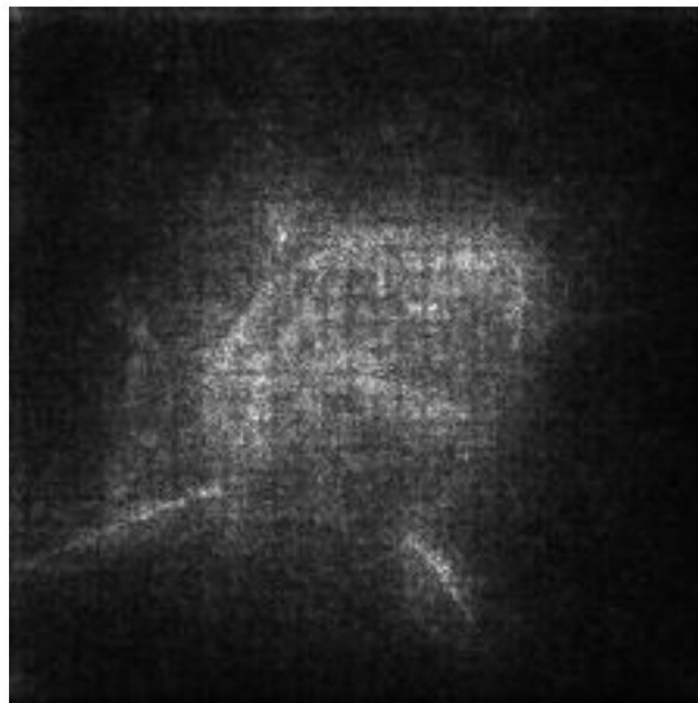
Lo que se hace es calcular las derivadas parciales de la salida de la clase a la que pertenece la imagen, en función de los píxeles de la imagen de entrada.



# Saliency maps

Deep Inside Convolutional Networks: Visualising  
Image Classification Models and Saliency Maps

Karen Simonyan    Andrea Vedaldi    Andrew Zisserman  
Visual Geometry Group, University of Oxford  
{karen, vedaldi, az}@robots.ox.ac.uk



Los Saliency Maps sobre los píxeles es una linda idea (sobre todo la de pensar la red en función de los píxeles y no de los pesos) pero lo que termina resaltando es bastante ruidoso y muchas veces no difiere mucho de un detector de bordes porque mira cada pixel por separado.

La técnica de Grad-CAM en vez de mirar los píxeles busca ver cómo los patches están votando para que una imagen sea de una determinada clase.

## Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization

Ramprasaath R. Selvaraju · Michael Cogswell · Abhishek Das · Ramakrishna Vedantam · Devi Parikh · Dhruv Batra

### 3 Grad-CAM

A number of previous works have asserted that deeper representations in a CNN capture higher-level visual constructs [6, 41]. Furthermore, convolutional layers naturally retain spatial information which is lost in fully-connected layers, so we can expect the last convolutional layers to have the best compromise between high-level semantics and detailed spatial information. The neurons in these layers look for semantic class-specific information in the image (say object parts). Grad-CAM uses the gradient information flowing into the last convolutional layer of the CNN to assign importance values to each neuron for a particular decision of interest. Although our technique is fairly general in that it can be used to explain activations in any layer of a deep network, in this work, we focus on explaining output layer decisions only.

As shown in Fig. 2, in order to obtain the class-discriminative localization map Grad-CAM  $L_{\text{Grad-CAM}}^c \in \mathbb{R}^{u \times v}$  of width  $u$  and height  $v$  for any class  $c$ , we first compute the gradient of the score for class  $c$ ,  $y^c$  (before the softmax), with respect to feature map activations  $A^k$  of a convolutional layer, i.e.  $\frac{\partial y^c}{\partial A^k}$ . These gradients flowing back are global-average-pooled <sup>2</sup> over the width and height dimensions (indexed by  $i$  and  $j$  respectively) to obtain the neuron importance weights  $\alpha_k^c$ :

$$\alpha_k^c = \overbrace{\frac{1}{Z} \sum_i \sum_j}^{\text{global average pooling}} \underbrace{\frac{\partial y^c}{\partial A_{ij}^k}}_{\text{gradients via backprop}} \quad (1)$$

# Grad-CAM Perros vs Gatos

Grad-CAM Interpretations (Red=Incorrect, Green=Correct)

Pred: Cat (Actual: Cat)



Pred: Dog (Actual: Dog)



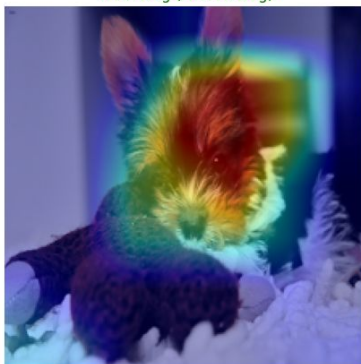
Pred: Dog (Actual: Dog)



Pred: Dog (Actual: Dog)



Pred: Dog (Actual: Dog)

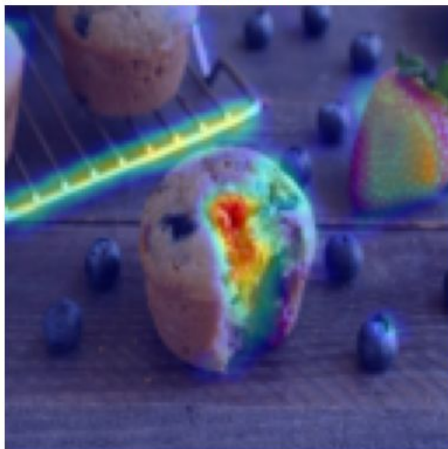


Pred: Cat (Actual: Cat)

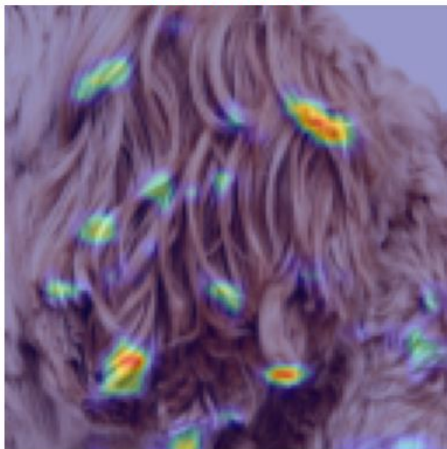


# Grad-CAM Chihuahuas vs Muffins

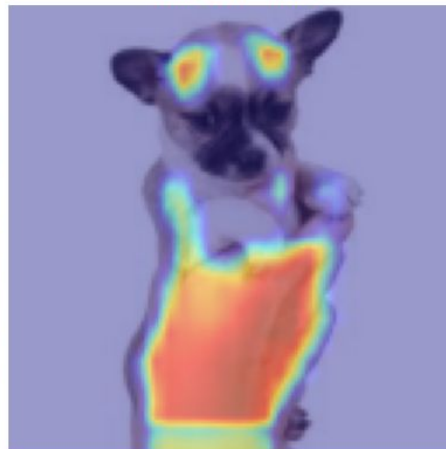
Pred: muffin  
Real: muffin



Grad-CAM: ¿Qué mira la red para decidir?  
Pred: muffin  
Real: chihuahua



Pred: chihuahua  
Real: chihuahua



Pred: muffin  
Real: muffin



# Modificando píxeles

La interpretabilidad nos ayuda no solo a entender qué está pasando, si no también a entender qué podemos hacer con el formalismo.

En los últimos métodos vimos que podemos pensar la red como una función con respecto a los píxeles en vez de con respecto a los pesos. Por ahora lo usamos *solo* para calcular gradientes, pero por qué no podríamos hacer al igual que hicimos con los pesos y usar esos gradientes para modificar los píxeles en pos de algún objetivo interesante.

# Atacando a la red

Si tengo una foto de un Chihuahua, que la red la clasifica bien, ¿qué tanto tengo que cambiarla para que la red crea que es un muffin?

Tal vez poco, dado que son dos clases parecidas. Si tengo una imagen de una pelota de tenis, que una VGG-16 clasifica bien, ¿qué tanto tengo que cambiarla para que la red crea que es un koala?

Podemos pensar en el siguiente flujo:

- Tomamos la imagen de la pelota de tenis y la metemos en la red.
- Elegimos la nueva clase (koala).
- Pensando la probabilidad de ser koala con esos pesos fijos y en función de los píxeles, vamos modificando la imagen para maximizar la probabilidad.



# Atacando a la red

Predicción original: 852 (Tenis Ball es 852)

Ataque  $\epsilon=0.03$ : 852 (Koala es 105)

--- Guardando la imagen adversarial ---

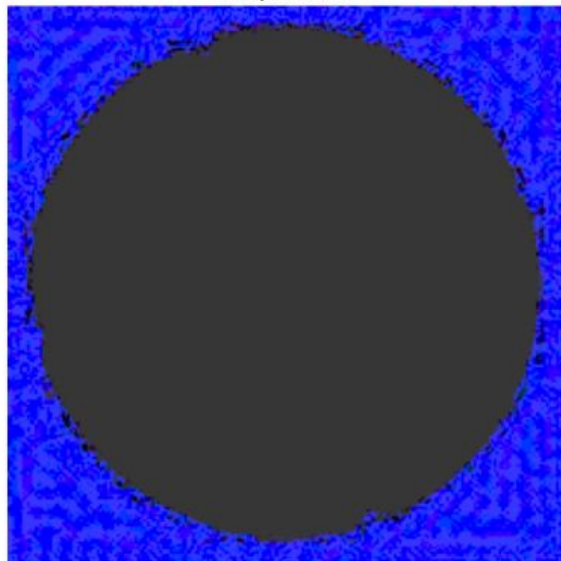
Imagen guardada como 'tennis\_ball\_fake\_koala.jpg'

--- Visualizando la diferencia ---

Original (Pelota de Tenis)



Perturbación  $\epsilon$ : 0.03  
(Amplificada)



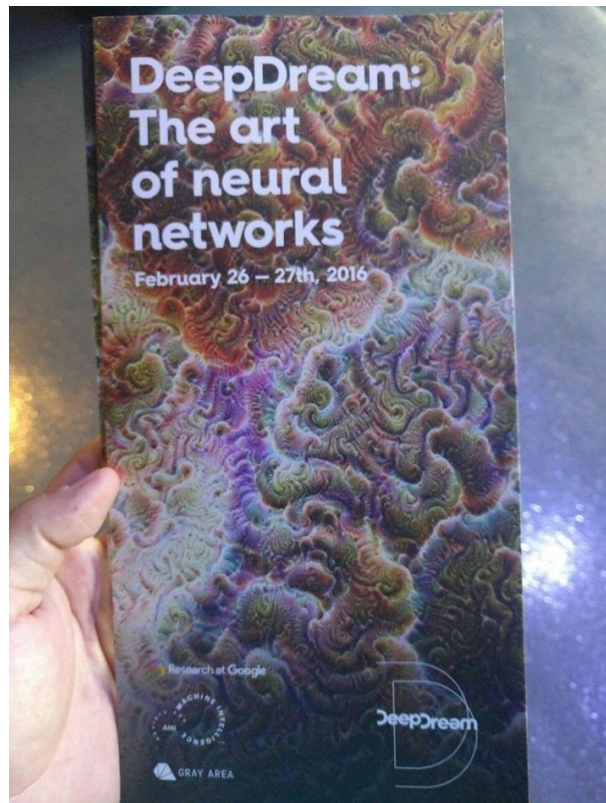
Adversarial (Koala fake)



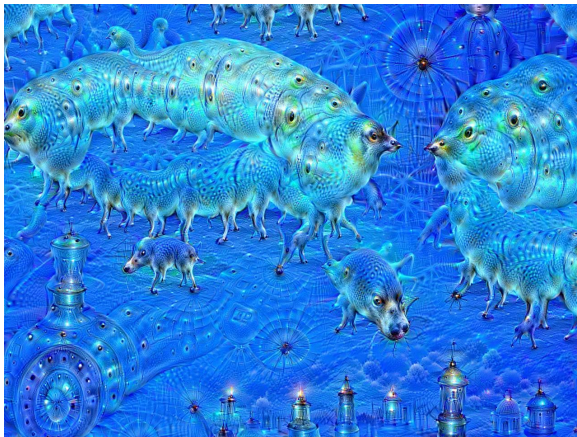
# Deep Dream

En el ataque presentado lo que maximizamos es la probabilidad de una clase arbitrariamente elegida. ¿Qué pasa si en vez de eso lo que maximizamos es la activación de ciertos filtros intermedios?

El proyecto DeepDream de Google buscó esto. El objetivo era casi puramente artístico pero también explica los *conceptos* que la red aprende y traza analogías con fenómenos como la *pareidolia*.



# Deep Dream



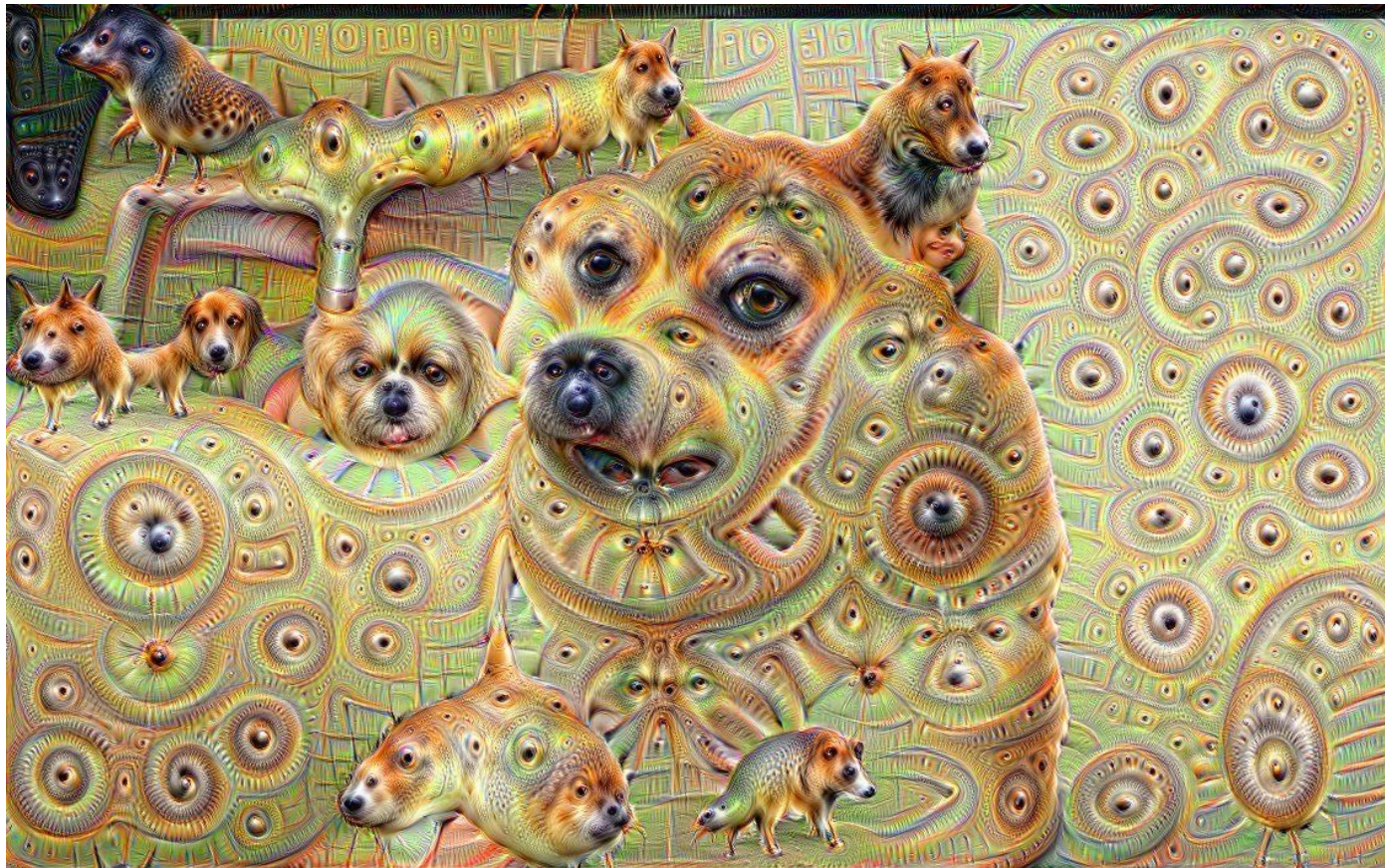


# Deep Dream





# Deep Dream



Vemos que podemos modificar la imagen persiguiendo diferentes objetivos. Si logramos plantear claramente los objetivos y entendemos qué hay que optimizar para lograrlo, podemos diseñar diferentes aplicaciones.

En el paper de Gatys et al, se busca determinar los objetivos correctos para traducir el *estilo* de una imagen en otra.

## A Neural Algorithm of Artistic Style

Leon A. Gatys,<sup>1,2,3\*</sup> Alexander S. Ecker,<sup>1,2,4,5</sup> Matthias Bethge<sup>1,2,4</sup>

<sup>1</sup>Werner Reichardt Centre for Integrative Neuroscience

and Institute of Theoretical Physics, University of Tübingen, Germany

<sup>2</sup>Bernstein Center for Computational Neuroscience, Tübingen, Germany

<sup>3</sup>Graduate School for Neural Information Processing, Tübingen, Germany

<sup>4</sup>Max Planck Institute for Biological Cybernetics, Tübingen, Germany

<sup>5</sup>Department of Neuroscience, Baylor College of Medicine, Houston, TX, USA

\*To whom correspondence should be addressed; E-mail: [leon.gatys@bethgelab.org](mailto:leon.gatys@bethgelab.org)



# Style Transfer

En el problema de Style Transfer se busca combinar 2 imágenes de tal manera que se preserve el *contenido* de una, mientras se preserve el *estilo* de la otra.

Vamos entonces a optimizar una función objetivo que tenga en cuenta estas dos cuestiones. El *content loss* es el más intuitivo dado que no es más que tomar alguna métrica de semejanza como el MSE con la imagen target.

El *style loss* es el novedoso de este trabajo. El *style loss* se define mediante una matriz de Gram que caracteriza la correlación entre diferentes filtros. Un estilo no es que cierta parte de la imagen tenga un trazo diagonal, si no que es la correlación de ciertos tipos de filtros en diferentes partes de la imagen (por ejemplo, Starry Night tiene remolinos de tonos celestes o amarillos).

El problema a resolver es entonces que el contenido se parezca a la imagen target de contenido, mientras que la matriz de gram de estilos se parezca a la matriz de gram de estilos de la imagen target para estilo.

To generate a texture that matches the style of a given image (Fig 1, style reconstructions), we use gradient descent from a white noise image to find another image that matches the style representation of the original image. This is done by minimising the mean-squared distance between the entries of the Gram matrix from the original image and the Gram matrix of the image to be generated. So let  $\vec{a}$  and  $\vec{x}$  be the original image and the image that is generated and  $A^l$  and  $G^l$  their respective style representations in layer  $l$ . The contribution of that layer to the total loss is then

$$E_l = \frac{1}{4N_l^2 M_l^2} \sum_{i,j} (G_{ij}^l - A_{ij}^l)^2 \quad (4)$$

and the total loss is

$$\mathcal{L}_{style}(\vec{a}, \vec{x}) = \sum_{l=0}^L w_l E_l \quad (5)$$

# Style Transfer

```
optimizer = optim.Adam([generated_img], lr=0.02)
```



# Generación de imágenes

Para cerrar nuestro camino con las imágenes, veamos un poco sobre el tema con más *hype* del momento, la generación.

En realidad, ya venimos generando imágenes con las técnicas vistas, pero en ciertos contextos muy controlados.

En procesamiento de texto vamos a ver que los problemas de predicción y generación son muy cercanos. En imágenes es un poco más difícil adentrarse en este mundo porque a priori la generación dista como problema de los que venimos tratando (clasificación, detección de objetos, segmentación).



# Generación de imágenes

En primer lugar para hablar de generación de imágenes hace falta hacerse algunas preguntas centrales. ¿Qué es generar una imagen? ¿A *partir de qué* genero una imagen nueva?

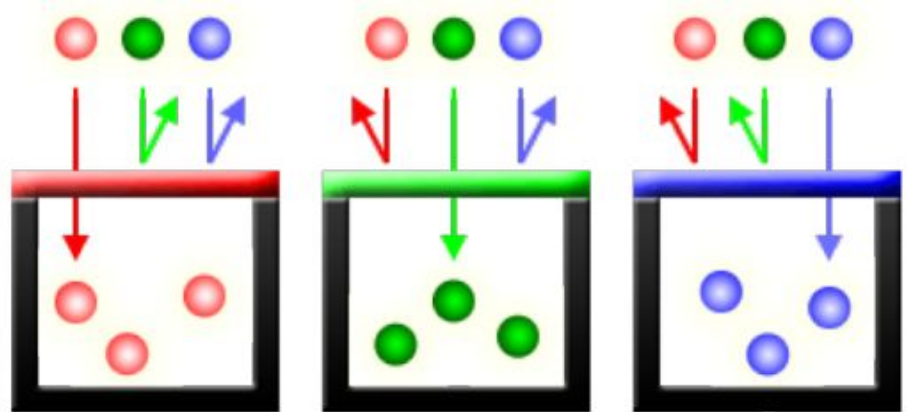
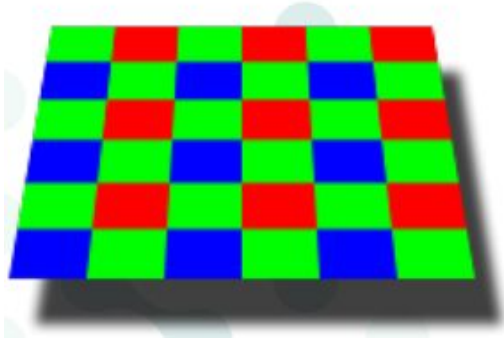
Hoy asociamos el proceso de generar una imagen a escribir un texto y obtener una imagen de esa descripción pero no tiene que ser necesariamente así.

Existen diversas maneras de generar imágenes:

- A partir de una imagen que le falta información (demosaiicing, inpainting).
- A partir de un dataset de imágenes.
- Combinando dos imágenes como en el caso de Style Transfer (Controlnet para pose)

# Generación de imágenes *prehistóricas*

La generación de imágenes tiene más años que las CNNs. Por ejemplo, el problema de Demosaicing en las cámaras fotográficas producen una imagen a partir de información parcial.



¿Cómo podemos agarrar alguna de las arquitecturas vistas y utilizarla para que generen nuevas imágenes?

# Generación Moderna *naive*

Si tengo un buen *autoencoder* entrenado puedo tirar el encoder, quedarme con el decoder y usarlo para generar.

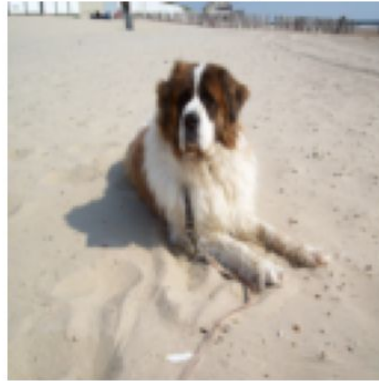
- Entrenamos un autoencoder (o tomamos uno preentrenado).
- Lo cortamos y nos quedamos con el decoder.
- Generamos vectores random (o con algún criterio) en el espacio latente.
- Los pasamos por el decoder y generamos una imagen.

# Generación Moderna naive

Perro A (Real)



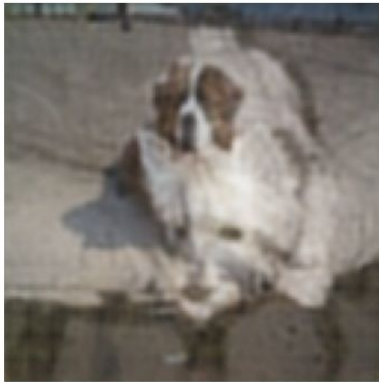
Perro B (Real)



Reconstrucción de A



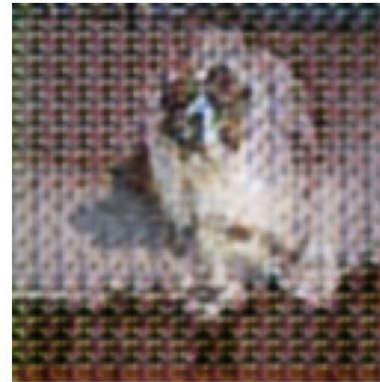
Test 1: Promedio (Fantasma)



Test 2: Escalar x2.5 (Ruido)



Test 3: Crossover (Frankenstein)





# Variational AutoEncoder

La generación naive tenía algún sentido pero no salió muy bien porque el espacio latente del autoencoder no está pensado para eso. Nunca fue entrenado para que las muestras guarden cierta cohesión entre sí.

La creación de los Variational AutoEncoder fue justamente para resolver este problema. Además de buscar la minimización clásica de un autoencoder también busca que el espacio latente guarde cohesión entre las muestras y que las representaciones no sean puntos sueltos en medio de un espacio de dimensión alta.

Durante algunos años el mundo de la generación de imágenes fue dominado principalmente por las arquitecturas tipo GAN (Generative Adversarial Network).

Se entrenan dos redes al mismo tiempo, una red que genera datos y otra red debe decidir si ese dato es real o generado.

---

## Generative Adversarial Nets

---

Ian J. Goodfellow, Jean Pouget-Abadie,<sup>\*</sup> Mehdi Mirza, Bing Xu, David Warde-Farley,  
Sherjil Ozair,<sup>†</sup> Aaron Courville, Yoshua Bengio<sup>‡</sup>  
Département d'informatique et de recherche opérationnelle  
Université de Montréal  
Montréal, QC H3C 3J7

### Abstract

We propose a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models: a generative model  $G$  that captures the data distribution, and a discriminative model  $D$  that estimates the probability that a sample came from the training data rather than  $G$ . The training procedure for  $G$  is to maximize the probability of  $D$  making a mistake. This framework corresponds to a minimax two-player game. In the space of arbitrary functions  $G$  and  $D$ , a unique solution exists, with  $G$  recovering the training data distribution and  $D$  equal to  $\frac{1}{2}$  everywhere. In the case where  $G$  and  $D$  are defined by multilayer perceptrons, the entire system can be trained with backpropagation. There is no need for any Markov chains or unrolled approximate inference networks during either training or generation of samples. Experiments demonstrate the potential of the framework through qualitative and quantitative evaluation of the generated samples.

Uno de los cambios más importantes en la generación de imágenes llegó con los *modelos de difusión*.

La idea central es ir sumando iterativamente un ruido particular a una imagen hasta que sea completamente ruido *estático*.

Luego se entrena una U-Net (la de segmentación) para aprender a deshacer el ruido de cada paso.

---

## Denoising Diffusion Probabilistic Models

---

Jonathan Ho  
UC Berkeley  
jonathanho@berkeley.edu

Ajay Jain  
UC Berkeley  
ajayj@berkeley.edu

Pieter Abbeel  
UC Berkeley  
pabbeel@cs.berkeley.edu

### Abstract

We present high quality image synthesis results using diffusion probabilistic models, a class of latent variable models inspired by considerations from nonequilibrium thermodynamics. Our best results are obtained by training on a weighted variational bound designed according to a novel connection between diffusion probabilistic models and denoising score matching with Langevin dynamics, and our models naturally admit a progressive lossy decompression scheme that can be interpreted as a generalization of autoregressive decoding. On the unconditional CIFAR10 dataset, we obtain an Inception score of 9.46 and a state-of-the-art FID score of 3.17. On 256x256 LSUN, we obtain sample quality similar to ProgressiveGAN. Our implementation is available at <https://github.com/hojonathanho/diffusion>.

DDPM fue una idea muy importante pero en la práctica era muy difícil de utilizar por su costo computacional.

El modelo DDIM llegó para plantear un entrenamiento análogo al de DDPM pero con un procedimiento mucho más eficiente, haciendo que sea utilizable en aplicaciones prácticas.

## DENOISING DIFFUSION IMPLICIT MODELS

Jiaming Song, Chenlin Meng & Stefano Ermon

Stanford University

{tsong, chenlin, ermon}@cs.stanford.edu

### ABSTRACT

Denoising diffusion probabilistic models (DDPMs) have achieved high quality image generation without adversarial training, yet they require simulating a Markov chain for many steps in order to produce a sample. To accelerate sampling, we present denoising diffusion implicit models (DDIMs), a more efficient class of iterative implicit probabilistic models with the same training procedure as DDPMs. In DDPMs, the generative process is defined as the reverse of a particular Markovian diffusion process. We generalize DDPMs via a class of non-Markovian diffusion processes that lead to the same training objective. These non-Markovian processes can correspond to generative processes that are deterministic, giving rise to implicit models that produce high quality samples much faster. We empirically demonstrate that DDIMs can produce high quality samples  $10\times$  to  $50\times$  faster in terms of wall-clock time compared to DDPMs, allow us to trade off computation for sample quality, perform semantically meaningful image interpolation directly in the latent space, and reconstruct observations with very low error.

# Stable Diffusion

El siguiente gran paso fue el trabajo seminal de Stable Diffusion que permitía realizar el proceso de difusión no sobre toda la imagen sino sobre el espacio latente de features. De esta manera el procesamiento pasó a ser notoriamente menos demandante (sobre todo a nivel memoria), lo que permitió el uso del modelo en computadoras de uso diario.

## High-Resolution Image Synthesis with Latent Diffusion Models

Robin Rombach<sup>1</sup> \*   Andreas Blattmann<sup>1</sup> \*   Dominik Lorenz<sup>1</sup>   Patrick Esser<sup>RS</sup>   Björn Ommer<sup>1</sup>

<sup>1</sup>Ludwig Maximilian University of Munich & IWR, Heidelberg University, Germany   <sup>RS</sup>Runway ML

<https://github.com/CompVis/latent-diffusion>

### Abstract

By decomposing the image formation process into a sequential application of denoising autoencoders, diffusion models (DMs) achieve state-of-the-art synthesis results on image data and beyond. Additionally, their formulation allows for a guiding mechanism to control the image generation process without retraining. However, since these models typically operate directly in pixel space, optimization of powerful DMs often consumes hundreds of GPU days and inference is expensive due to sequential evaluations. To enable DM training on limited computational resources while retaining their quality and flexibility, we apply them in the latent space of powerful pretrained autoencoders. In contrast to previous work, training diffusion models on such a representation allows for the first time to reach a near-optimal point between complexity reduction and detail preservation, greatly boosting visual fidelity. By introducing cross-attention layers into the model architecture, we turn diffusion models into powerful and flexible generators for general conditioning inputs such as text or bounding boxes and high-resolution synthesis becomes possible in a convolutional manner. Our latent diffusion models (LDMs) achieve new state-of-the-art scores for image inpainting and class-conditional image synthesis and highly competitive performance on various tasks, including text-to-image synthesis, unconditional image generation and super-resolution, while significantly reducing computational requirements compared to pixel-based DMs.

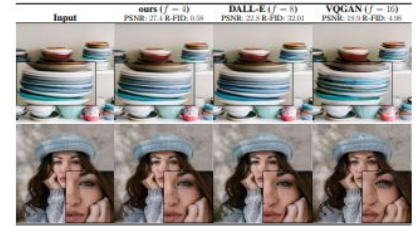


Figure 1. Boosting the upper bound on achievable quality with less aggressive downsampling. Since diffusion models offer excellent inductive biases for spatial data, we do not need the heavy spatial downsampling of related generative models in latent space, but can still greatly reduce the dimensionality of the data via suitable autoencoding models, see Sec. 3. Images are from the DIV2K [11] validation set, evaluated at  $512^2$  px. We denote the spatial downsampling factor by  $f$ . Reconstruction FIDs [29] and PSNR are calculated on ImageNet-val. [12]; see also Tab. 8.

results in image synthesis [30,85] and beyond [7,45,48,57], and define the state-of-the-art in class-conditional image synthesis [15,31] and super-resolution [72]. Moreover, even unconditional DMs can readily be applied to tasks such as inpainting and colorization [85] or stroke-based synthesis [53], in contrast to other types of generative models [19,46,69]. Being likelihood-based models, they do not exhibit mode-collapse and training instabilities as GANs and, by heavily exploiting parameter sharing, they can



Todas las arquitecturas que mencionamos son para generar imágenes a partir de una cierta red entrenada con tal fin.

El paper de CLIP fue uno de los trabajos más importantes en el camino de la *multimodalidad* que une el texto con las imágenes.

---

## Learning Transferable Visual Models From Natural Language Supervision

---

Alec Radford<sup>\*1</sup> Jong Wook Kim<sup>\*1</sup> Chris Hallacy<sup>1</sup> Aditya Ramesh<sup>1</sup> Gabriel Goh<sup>1</sup> Sandhini Agarwal<sup>1</sup>  
 Girish Sastry<sup>1</sup> Amanda Askell<sup>1</sup> Pamela Mishkin<sup>1</sup> Jack Clark<sup>1</sup> Gretchen Krueger<sup>1</sup> Ilya Sutskever<sup>1</sup>

### Abstract

State-of-the-art computer vision systems are trained to predict a fixed set of predetermined object categories. This restricted form of supervision limits their generality and usability since additional labeled data is needed to specify any other visual concept. Learning directly from raw text about images is a promising alternative which leverages a much broader source of supervision. We demonstrate that the simple pre-training task of predicting which caption goes with which image is an efficient and scalable way to learn SOTA image representations from scratch on a dataset of 400 million (image, text) pairs collected from the internet. After pre-training, natural language is used to reference learned visual concepts (or describe new ones) enabling zero-shot transfer of the model to downstream tasks. We study the performance of this approach by benchmarking on over 30 different existing computer vision datasets, spanning tasks such as OCR, action recognition in videos, geo-localization, and many types of fine-grained object classification. The model transfers non-trivially to most tasks and is often competitive with a fully supervised baseline without the need for any dataset specific training. For instance, we match the accuracy of the original ResNet-50 on ImageNet zero-shot without needing to use any of the 1.28 million training examples it was trained on. We release our code and pre-trained model weights at <https://github.com/OpenAI/CLIP>.

Task-agnostic objectives such as autoregressive and masked language modeling have scaled across many orders of magnitude in compute, model capacity, and data, steadily improving capabilities. The development of “text-to-text” as a standardized input-output interface (McCann et al., 2018; Radford et al., 2019; Raffel et al., 2019) has enabled task-agnostic architectures to zero-shot transfer to downstream datasets removing the need for specialized output heads or dataset specific customization. Flagship systems like GPT-3 (Brown et al., 2020) are now competitive across many tasks with bespoke models while requiring little to no dataset specific training data.

These results suggest that the aggregate supervision accessible to modern pre-training methods within web-scale collections of text surpasses that of high-quality crowd-labeled NLP datasets. However, in other fields such as computer vision it is still standard practice to pre-train models on crowd-labeled datasets such as ImageNet (Deng et al., 2009). Could scalable pre-training methods which learn directly from web text result in a similar breakthrough in computer vision? Prior work is encouraging.

Over 20 years ago Mori et al. (1999) explored improving content based image retrieval by training a model to predict the nouns and adjectives in text documents paired with images. Quattoni et al. (2007) demonstrated it was possible to learn more data efficient image representations via manifold learning in the weight space of classifiers trained to predict words in captions associated with images. Srivastava & Salakhutdinov (2012) explored deep representation learning by training multimodal Deep Boltzmann Machines on top of low-level image and text tag features. Joulin et al. (2016) modernized this line of work and demonstrated that CNNs trained to predict words in image captions

# Vision Transformers

El siguiente gran salto en el mundo de las imágenes (tanto en generación como en el resto de los problemas) lo están dando los Vision Transformers (ViT).

Es una arquitectura novedosa que viene del mundo del procesamiento de secuencias y luego fue adaptado al mundo de las imágenes.

## AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy<sup>\*,†</sup>, Lucas Beyer<sup>\*</sup>, Alexander Kolesnikov<sup>\*</sup>, Dirk Weissenborn<sup>\*</sup>,  
Xiaohua Zhai<sup>\*</sup>, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,  
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby<sup>\*,†</sup>

<sup>\*</sup>equal technical contribution, <sup>†</sup>equal advising

Google Research, Brain Team

{adosovitskiy, neilhoulby}@google.com

### ABSTRACT

While the Transformer architecture has become the de-facto standard for natural language processing tasks, its applications to computer vision remain limited. In vision, attention is either applied in conjunction with convolutional networks, or used to replace certain components of convolutional networks while keeping their overall structure in place. We show that this reliance on CNNs is not necessary and a pure transformer applied directly to sequences of image patches can perform very well on image classification tasks. When pre-trained on large amounts of data and transferred to multiple mid-sized or small image recognition benchmarks (ImageNet, CIFAR-100, VTAB, etc.), Vision Transformer (ViT) attains excellent results compared to state-of-the-art convolutional networks while requiring substantially fewer computational resources to train.<sup>1</sup>